

HEAPS AND AVL TREES

- Introduction to Heaps
- Heap Sort
- Priority Queue using Heaps
- AVL Trees
 - Terminologies
 - Basic Operations
 - Rotation
 - Insertion
 - Deletion

AVL Trees

- AVL tree is a height-balanced binary search tree
- It is also a binary search tree but it is a balanced tree.
- **What is Balanced Tree?**
 - A binary tree is said to be balanced if, the difference between the heights of left and right subtrees of every node in the tree is either **-1, 0 or +1**
- In an AVL tree, every node maintains an extra information known as **balance factor**

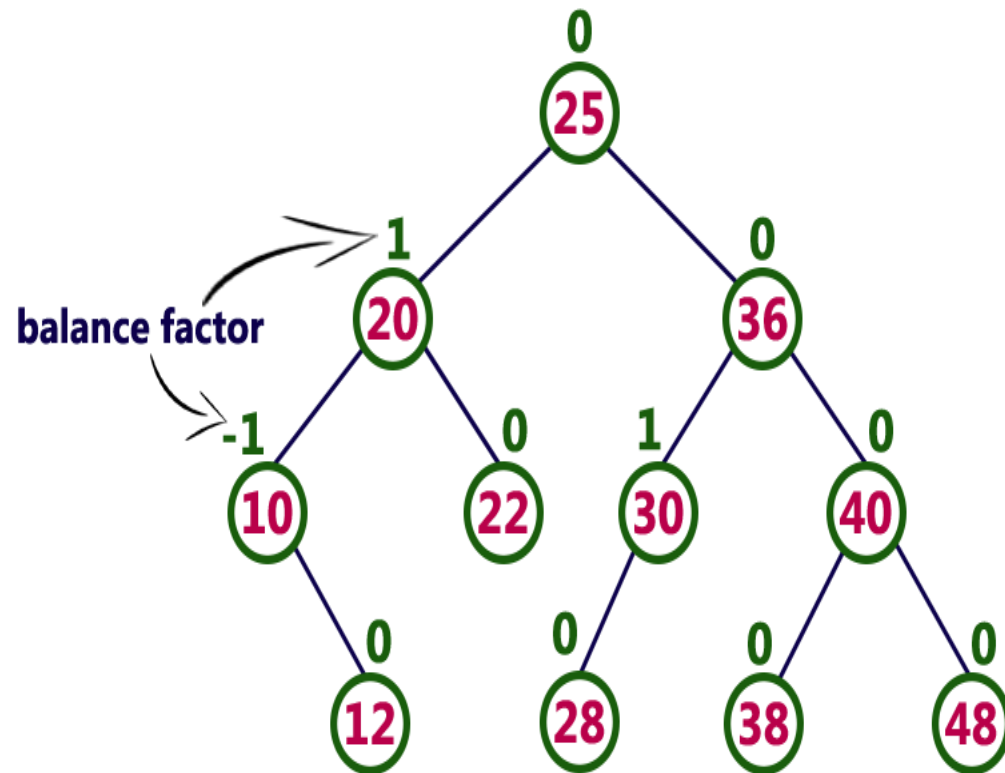
AVL Trees

- AVL tree was introduced by G.M. Adelson-Velsky and E.M. Landis
- Balance factor of a node is the difference between the heights of the left and right subtrees of that node.
- Balance factor = **height of left subtree - height of right subtree** (OR) **height of right subtree - height of left subtree**.

Why AVL Trees?

- AVL tree controls the height of the binary search tree by not letting it to be skewed.
- Time taken for all operations in BST= $O(h)$
- When it is Skewed it is $O(n)$, For limiting this, AVL tree is Introduced.
- For example : Searching is inefficient(i.e takes more time) in BST, When large number of nodes available in the tree when height is not balanced

Example of AVL Trees



Every AVL Tree is a binary search tree but every Binary Search Tree need not be AVL tree.

AVL Tree Rotations

- When performing operations like insertion and deletion - check the **balance factor** of every node in the tree.
- If not Balanced, What should be done??
 - **Rotation** operations is done to make the tree balanced
- What is Rotation??
 - Rotation is the process of moving nodes either to left or to right to make the tree balanced.

Classification of Rotations

1. Single Rotation

1. Left Rotation(LL Rotation)
2. Right Rotation(RR Rotation)

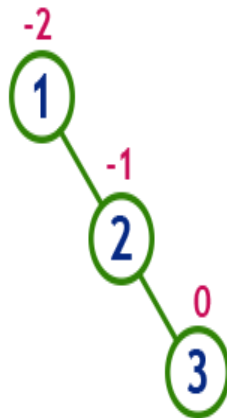
2. Double Rotation

1. Left Right Rotation(LR Rotation)
2. Right Left Rotation(RL Rotation)

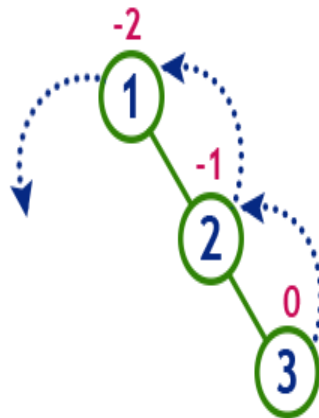
Single Left Rotation (LL Rotation)

- Every node moves one position to left from the current position

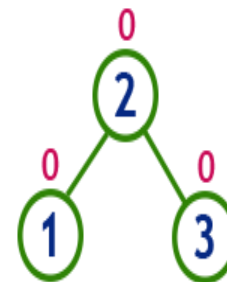
insert 1, 2 and 3



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left

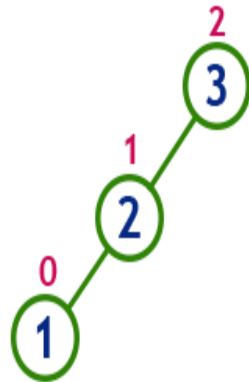


After LL Rotation Tree is Balanced

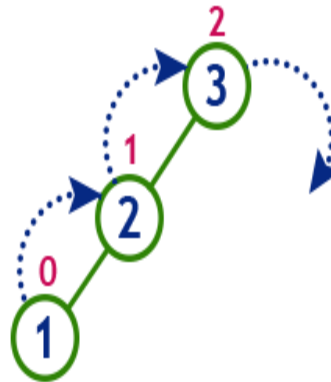
Single Right Rotation (RR Rotation)

- Every node moves one position to right from the current position

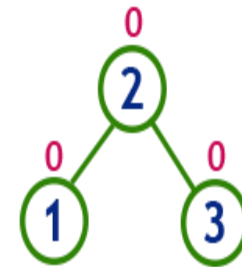
insert 3, 2 and 1



Tree is imbalanced
because node 3 has balance factor 2



To make balanced we use
RR Rotation which moves
nodes one position to right

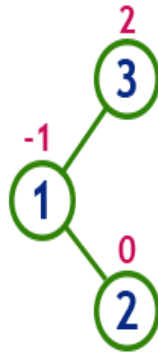


After RR Rotation
Tree is Balanced

Double Rotation- Left Right Rotation (LR Rotation)

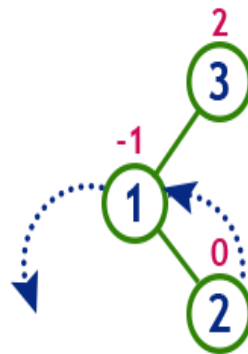
- Sequence of single left rotation followed by a single right rotation
- Every node moves one position to the left and one position to right from the current position

insert 3, 1 and 2



Tree is imbalanced

because node 3 has balance factor 2



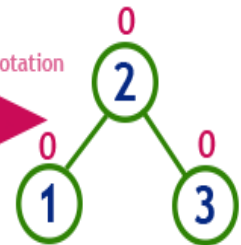
LL Rotation

After LL Rotation



RR Rotation

After RR Rotation



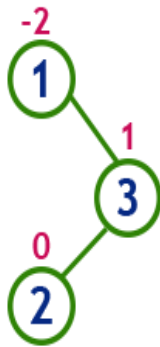
After LR Rotation

Tree is Balanced

Double Rotation- Right Left Rotation (RL Rotation)

- Sequence of single right rotation followed by a single left rotation
- Every node moves one position to the right and one position to left from the current position

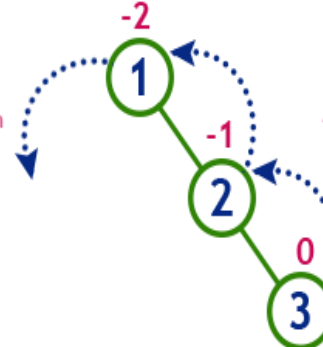
insert 1, 3 and 2



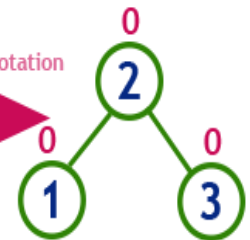
Tree is imbalanced
because node 1 has balance factor -2



RR Rotation



LL Rotation



**After RL Rotation
Tree is Balanced**

Operations on AVL Tree

- Insertion
- Search
- Deletion

Insertion Operation

- As in BST, a new node is always inserted as a leaf node.
- Step 1 - Insert the new element into the tree using Binary Search Tree insertion logic.
- Step 2 - After insertion, check the **Balance Factor of every node**.
- Step 3 - If the Balance Factor of every node is 0 or 1 or -1 then go for next operation.
- Step 4 - If the Balance Factor of any node is other than 0 or 1 or -1 then that tree is said to be imbalanced. In this case, perform suitable Rotation to make it balanced and go for next operation.

Construction of AVL Tree (1 to 8)

- Insert 1

$BF=0$



TREE is Balanced

- Insert 2

$BF=-1$



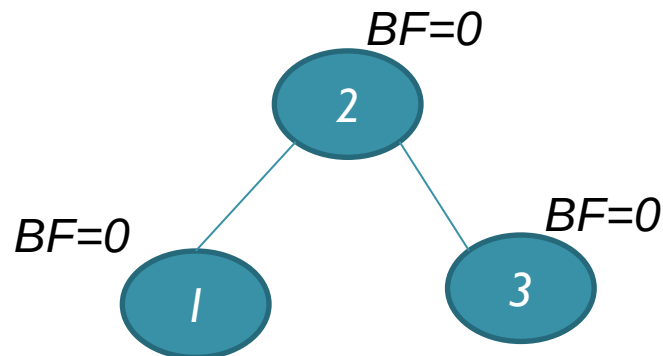
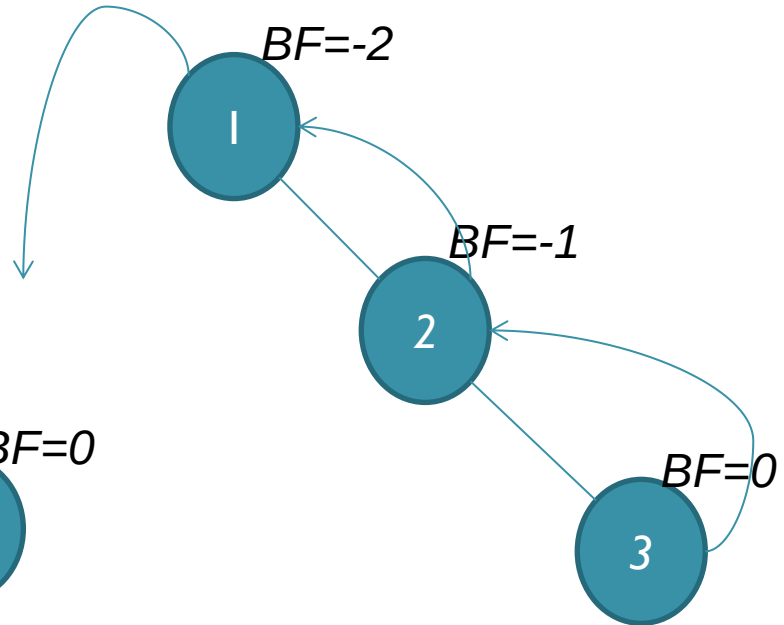
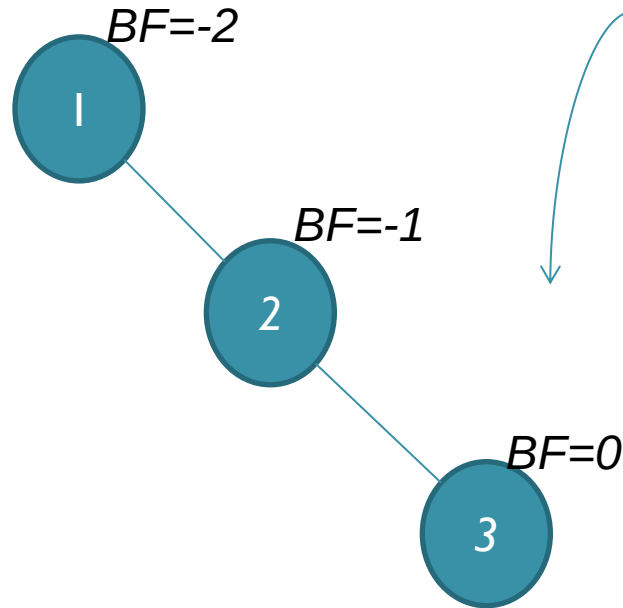
$BF=0$



TREE is Balanced

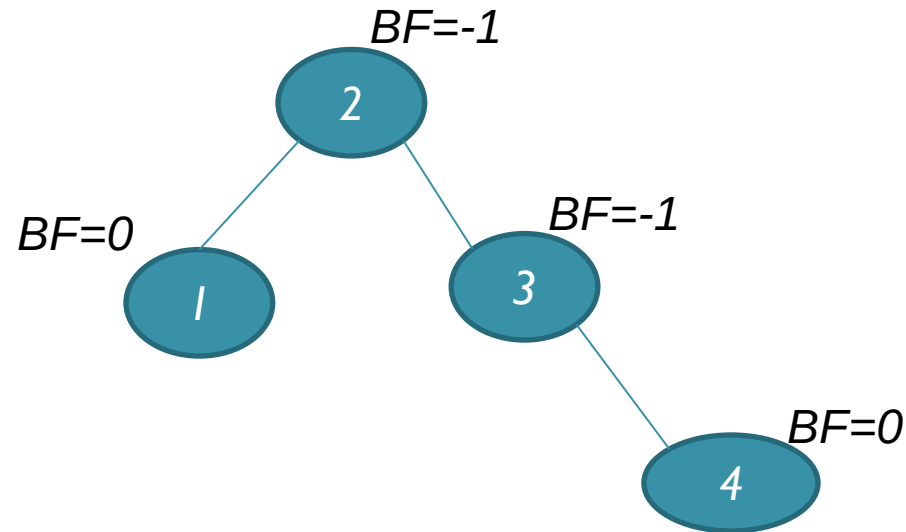
Construction of AVL Tree (1 to 8)

- Insert 3



Construction of AVL Tree (1 to 8)

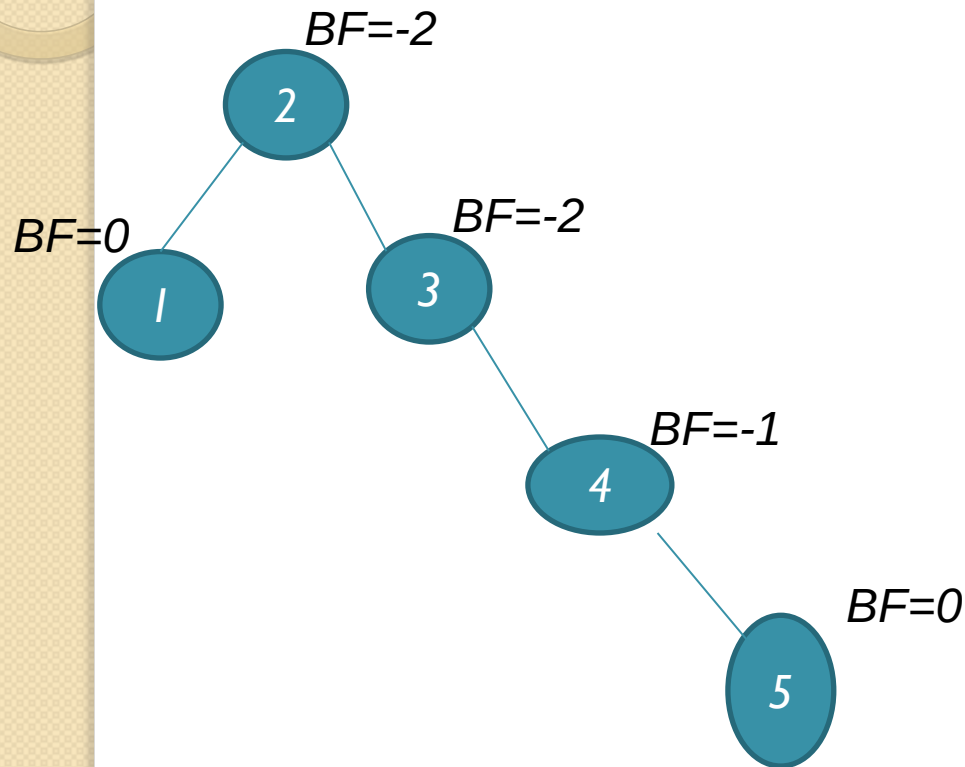
- Insert 4



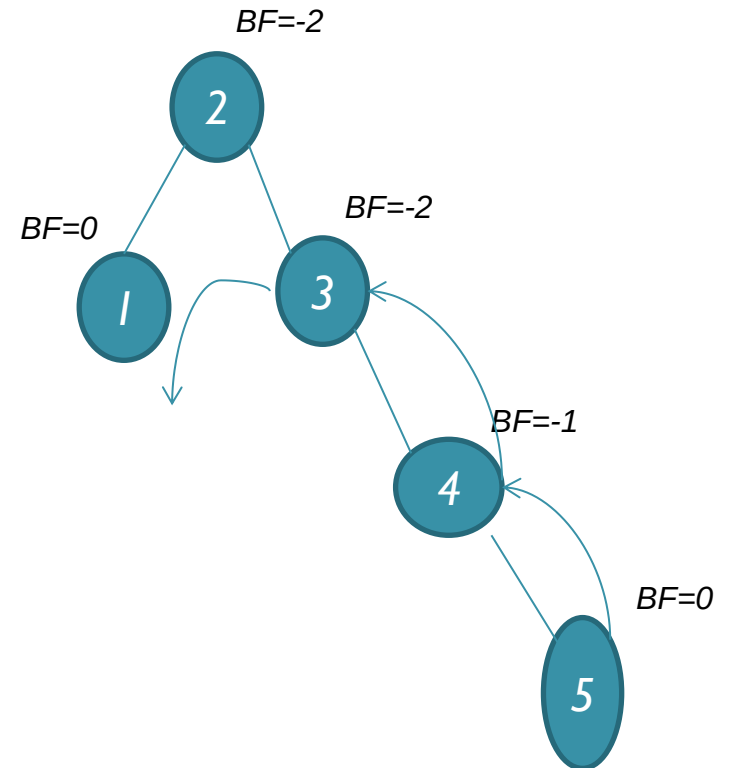
TREE is balanced

Construction of AVL Tree (1 to 8)

- Insert 5



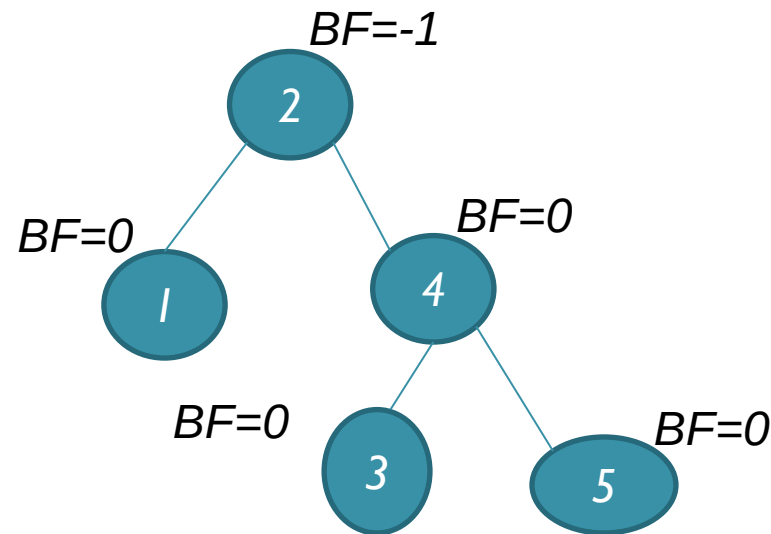
TREE is imbalanced



LL ROTATION AT 3

Construction of AVL Tree (1 to 8)

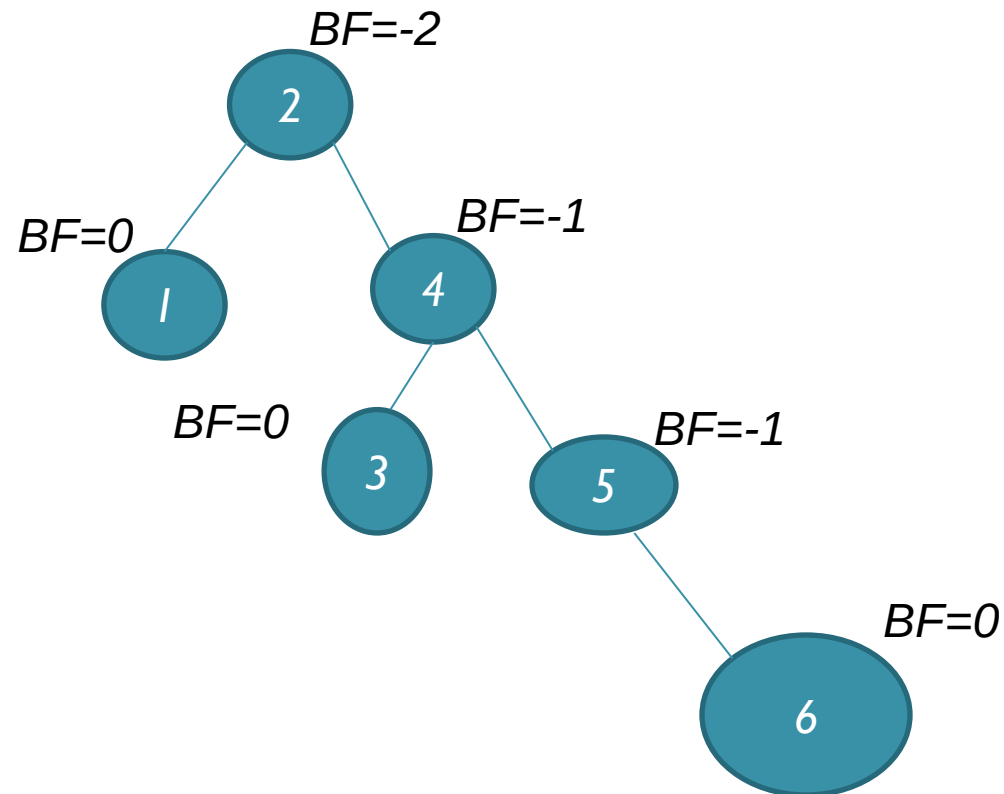
- Result of Inserting 5



TREE is balanced

Construction of AVL Tree (1 to 8)

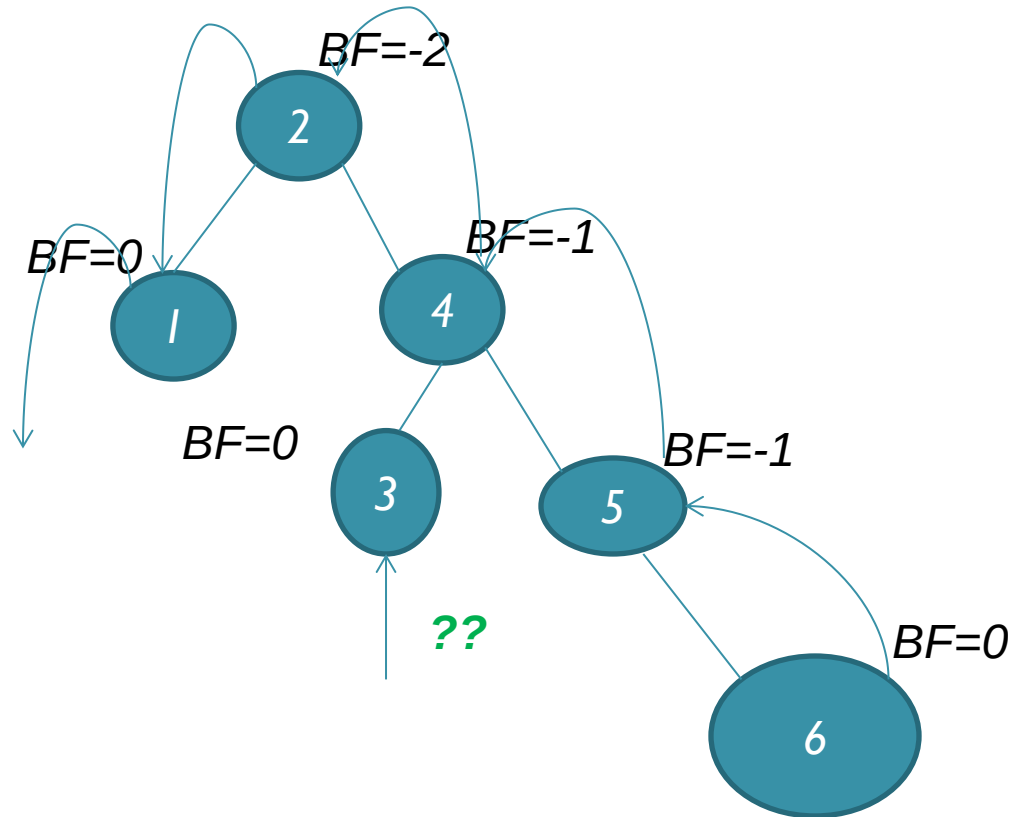
- Insert 6



TREE is imbalanced

Construction of AVL Tree (1 to 8)

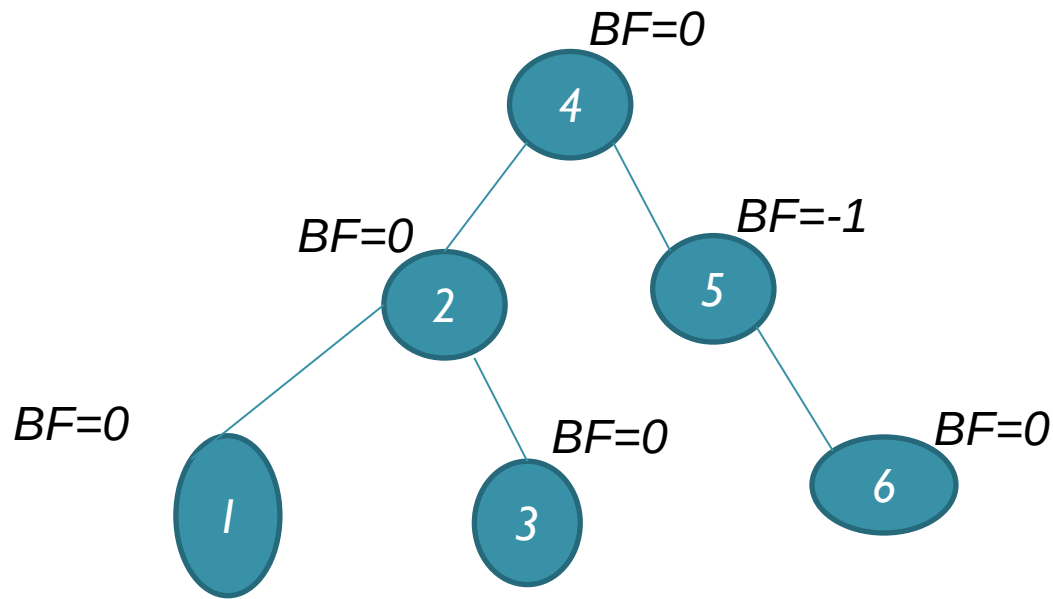
- Rotation process at Inserting 6



LL Rotation at 2

Construction of AVL Tree (1 to 8)

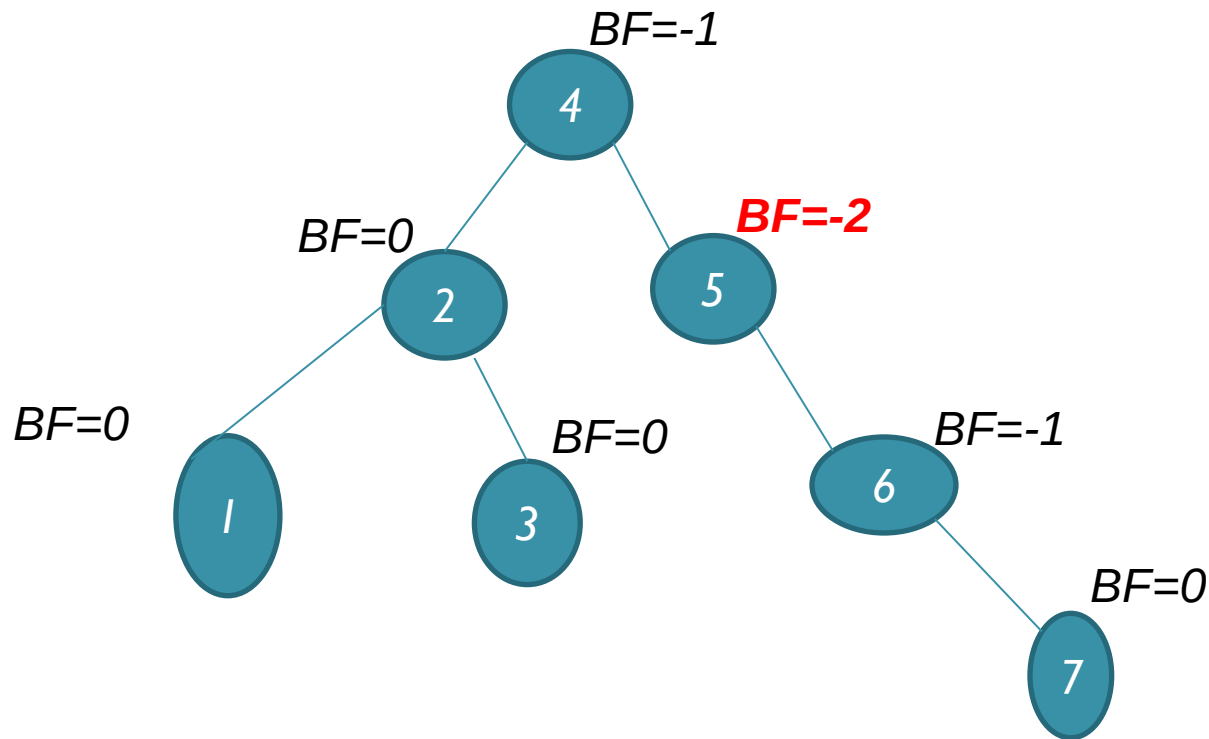
- Result of Inserting 6



TREE is balanced

Construction of AVL Tree (1 to 8)

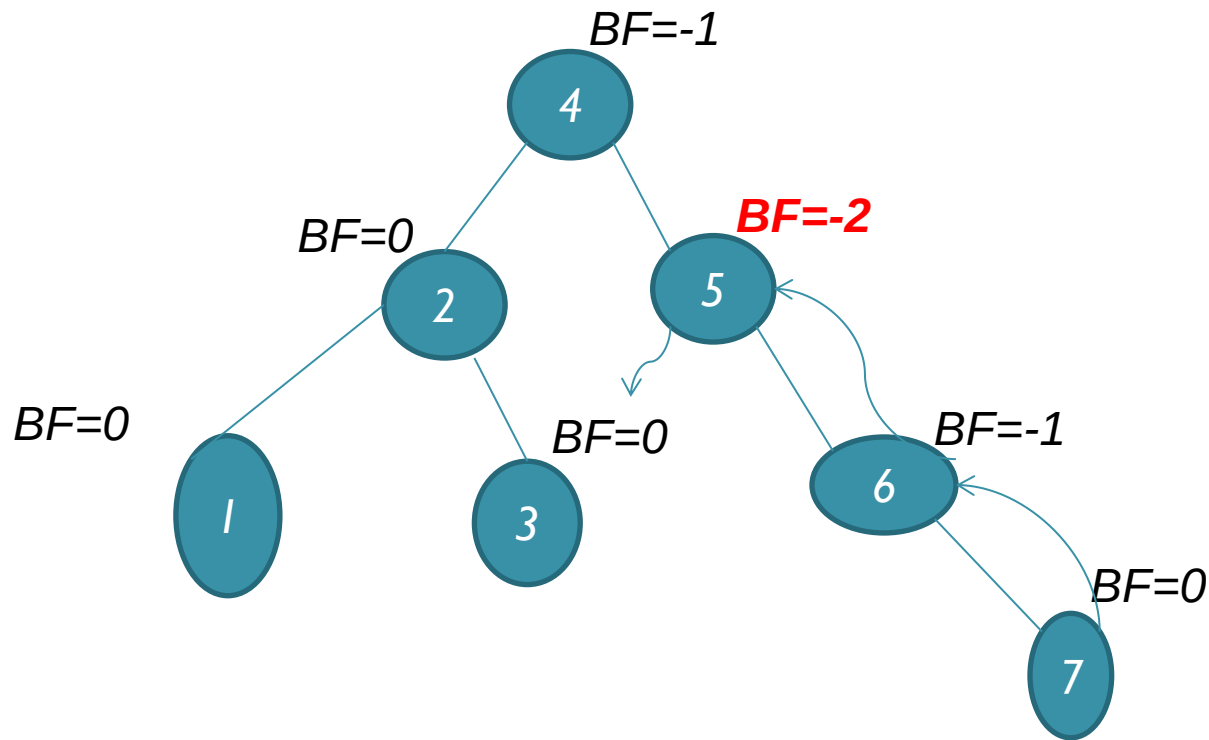
- Insert 7



TREE is imbalanced

Construction of AVL Tree (1 to 8)

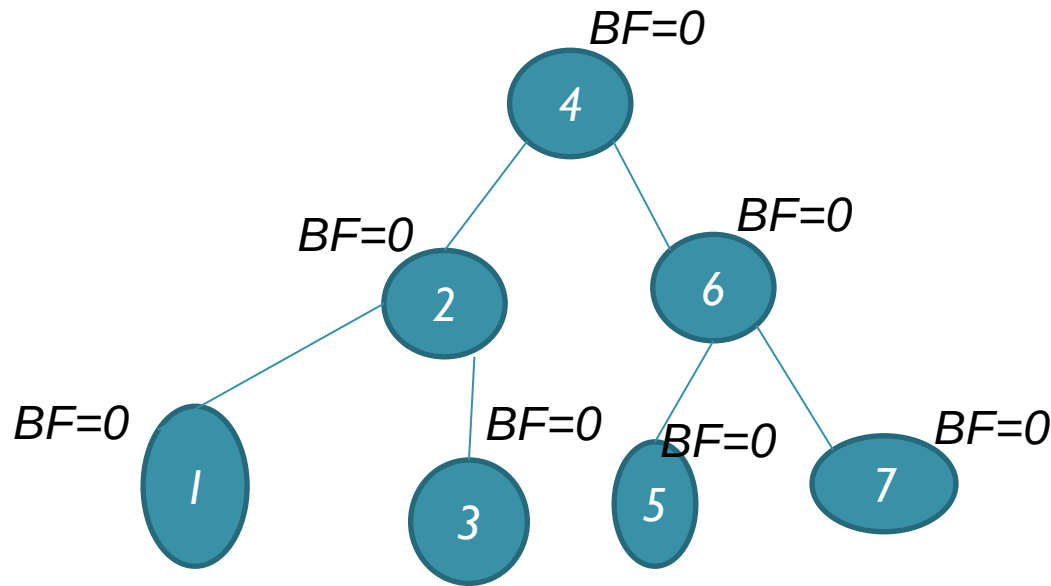
- Rotation Process when Inserting 7



LL Rotation at 5

Construction of AVL Tree (1 to 8)

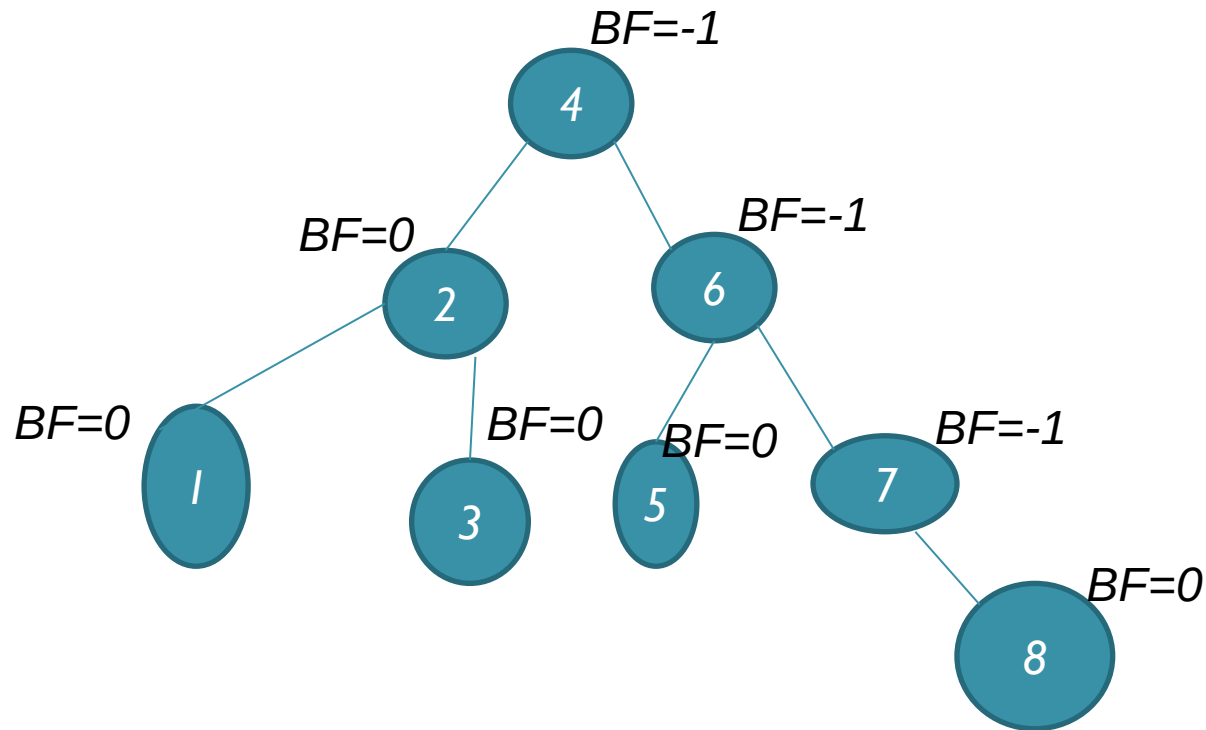
- Result of Inserting 7



TREE is balanced

Construction of AVL Tree (1 to 8)

- Insert 8



TREE is balanced

Deletion Operation

- Similar to deletion operation in BST
- Every deletion operation, we need to check with the Balance Factor condition
- If the tree is balanced after deletion go for next operation otherwise perform suitable rotation to make the tree Balanced.